Planning, Reasoning, & Architecture

The Agent's Brain

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Reasoning = Planning + Acting

- Reasoning is not just thinking!
  - Goal-directed planning
  - Task decomposition (eg. Plan and book a 3-day trip to Tokyo)
  - State tracking (what have I done? what is left to do?)
  - Adaptive Execution — The capacity to pause, observe real-world feedback, and adjust the plan (The Feedback Loop)
- Need carefully devised prompts that,
  - Define the Persona & Objective (The Goal)
  - Expose Available Tools/APIs (The Action Space)
  - Inject the "Scratchpad" of history (The State/Memory)
  - Enforce the Thought-Action-Observation format (The Loop)

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

A ReAct Agent

- Introduce the loop: Thought, Action, Observation.
  - Goal-driven loop that alternates brief reasoning with tool use.
  - Language is the glue: the agent explains (succinctly) what it needs, calls a tool, reads the result, then adapts.
  - Why it works: incremental feedback reduces hallucination and helps with planning under uncertainty.
  - Guardrails: cap steps/seconds, validate tool outputs, and stop on success criteria.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The ReAct Framework

```
Answer the following questions as best you can. You are a helpful assistant focused on accuracy.

You have access to the following tools:
{tools}

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of {tools}
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

Begin!

Question: {input}
Thought: {agent_scratchpad}
```


Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

The ReAct Prompt (Yao et al., 2022, LangChain)

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

An Example

Goal: “Email the team a lunch spot within 10 minutes’ walk, open now, avg price <$15.”

Step	Thought	Action	Observation
1	Need candidate restaurants nearby and open now.	WebSearch("cheap lunch open now near CS building")	3 candidates with hours & links.
2	Filter by walking time ≤10 minutes.	Maps.distance(candidate addrs, origin)	Two options within 8–10 minutes.
3	Check average prices (must be <$15).	Browser.fetch(menu pages)	One option averages $12–$14.
4	Pick best trade-off: distance + price + ratings.	— (decision step)	Chosen: “Taco Garden” (8 min, $13 avg).
5	Draft and send the email.	Email.send(to=team, body=details)	Email queued/sent; task complete.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Plan-and-Execute vs ReAct

- ReAct: You figure it out step-by-step. Good for high uncertainty. (Slow, Expensive, Flexible)
- Plan-and-Execute: ake the full list of steps before you start. Good for low uncertainty. (Rigid, Efficient, Fragile)

```
You are an expert Planner.
Your goal is to understand the user's request and create a step-by-step
plan to achieve it.
The plan should be a list of actionable steps.
Do not execute the steps yourself. Just list them.

User Request: {input}

Output Format:
1. Step 1
2. Step 2
3. ...
```

```
You are an expert Executor.

Your goal is to execute the current step of the plan.

# THE PLAN
{plan}

# RELEVANT CONTEXT
The following steps have already been completed:
{previous_steps}

# YOUR CURRENT TASK
Step: {current_step}

Execute this step and return the result.
```

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Wang et al., 2023, LangChain

Alternative Architectures

Approach	What it is (mental model)	When to use	Strengths	Gotchas
ReAct	Loop of **Think → Act → Observe → Repeat**. Uses a scratch pad built from action/observation pairs.	Exploratory tasks with uncertainty that benefit from incremental feedback.	Great intuition; reduces hallucination by grounding each step; flexible.	Can loop; needs **max steps/timeouts**; more tokens & latency.
Plan-and-Execute	Model first creates a **plan** (sub-steps), then executes steps (optionally with ReAct inside each).	Long-horizon tasks with clear decomposition (research, multi-API workflows).	Fewer loops; clearer progress; easier to audit ("here's the plan").	Brittle if initial plan is wrong; needs **re-planning** hooks; a bit more plumbing.
Tool-calling agent (single-turn)	One LLM call that may **choose & call tools** once, then answer. ("Function calling.")	Simple tasks with predictable I/O (translate file, fetch weather, call API, format output).	Fast, cheap, low loop risk; easy to test.	Limited adaptivity; no multi-step planning; brittle if one tool call isn't enough.
AutoGPT-style	Highly autonomous loop that **sets sub-goals, critiques, retries**, often with long-term memory.	Open-ended goals where human oversight is okay and cost/latency are secondary.	Powerful for discovery; can run unattended; showcases "agentic" flavor.	Unpredictable, costly; strong loop risk; heavy guardrails/memory curation needed.
LangGraph DAGs	**State-machine/graph**: nodes = tools/LLM steps, edges = routing/ conditions; ReAct is one possible subgraph.	Production flows needing reliability, observability, retries, branching, and tests.	Deterministic control; easy to add guardrails, fallbacks, concurrency; great traces.	More engineering up front; you design the rails (but that's the point!).

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

“Pick a lunch spot”

- ReAct: Search → see options → measure walking time → fetch menu → choose → email.
  Iterates with evidence each step; needs a recursion_limit.
- Plan-and-Execute: Plan: (1) find candidates, (2) filter by distance, (3) check price, (4) email. Then execute steps; re-plan if a step fails.
- Tool-calling agent: “Find a cheap lunch near me.” → calls Search API once → returns a guess.
  If wrong, you must ask again; no self-correction.
- AutoGPT-style: Sets sub-goals, saves notes to memory, tries different searches, maybe overthinks; needs strict budget/step caps.
- LangGraph DAGs: Nodes: search → filter_distance → check_price → decision → send_email; add edges for errors/timeouts; swap a node without changing the rest.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

System 1 vs. System 2

- Daniel Kahneman – Thinking, Fast and Slow
  - System 1: Fast / Intuitive, Effortless, always active
  - System 2: Slow / Deliberate, Effortful, requires attention and focus
- Self-Reflection & Critique
- Agents can hallucinate
  - Generate–Evaluate–Refine loop
  - Critic agents improve quality

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Multi-Agent Collaboration

- Specialized agents
  - Clear handoffs
  - Mirrors human teams
  - Handyman Vs The Crew (framer, plumber)
  - More specific objectives and guard rails
  - Modular
  - Example
    - Medical Analyst Agent
    - Safety and Compliance Agent
    - Patient Liaison Agent

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution

This file is meant for personal use by mpsii@mac.com only.
Sharing or publishing the contents in part or full is liable for legal action.